

# Documento de Apoio: Técnicas Essenciais de Pandas para Data, Limpar Dados e Exportar

## 1. Trabalhando com Dados Temporais em Pandas

### 1.1 Conversão de Strings para Datetime

```
python
import pandas as pd

# Conversão básica de datas
df['Data'] = pd.to_datetime(df['Data_Texto'])

# Para datas no formato brasileiro (DD/MM/YYYY)
df['Data'] = pd.to_datetime(df['Data_Texto'], dayfirst=True)

# Lidando com formatos mistos e erros
df['Data'] = pd.to_datetime(
    df['Data_Texto'],
    dayfirst=True,      # Primeiro número é o dia
    errors='coerce'     # Converte erros para NaT (Not a Time)
)
```

### 1.2 Extração de Componentes de Data

```
python
# Extraíndo diferentes componentes
df['Ano'] = df['Data'].dt.year
df['Mês'] = df['Data'].dt.month
df['Dia'] = df['Data'].dt.day
df['Hora'] = df['Data'].dt.hour

# Nomes dos dias e meses
df['Dia_Semana'] = df['Data'].dt.day_name()
df['Mês_Nome'] = df['Data'].dt.month_name()

# Outros componentes úteis
df['Trimestre'] = df['Data'].dt.quarter
```

```
df['Semana_Ano'] = df['Data'].dt.isocalendar().week
df['Dia_Ano'] = df['Data'].dt.dayofyear
```

## 1.3 Filtragem por Período Temporal

```
python
from datetime import datetime, timedelta

# Filtro por data específica
df_filtrado = df[df['Data'] >= '2024-01-01']

# Filtro dos últimos N dias
data_limite = datetime.now() - timedelta(days=30)
df_ultimos_30_dias = df[df['Data'] >= data_limite]

# Filtro por mês específico
df_janeiro = df[df['Data'].dt.month == 1]

# Filtro por trimestre
df_trimestre1 = df[df['Data'].dt.quarter == 1]

# Filtro por intervalo de datas
df_intervalo = df[(df['Data'] >= '2024-01-01') & (df['Data'] <= '2024-03-31')]
```

## 1.4 Cálculos com Datas

```
python
# Diferença entre datas
df['Dias_Diferenca'] = (df['Data_Final'] - df['Data_Inicial']).dt.days

# Adicionar dias a uma data
df['Data_Futura'] = df['Data'] + pd.Timedelta(days=7)

# Adicionar meses (usando DateOffset)
df['Data_6meses'] = df['Data'] + pd.DateOffset(months=6)

# Encontrar datas mínimas e máximas
data_minima = df['Data'].min()
data_maxima = df['Data'].max()
```

## 1.5 Agrupamento por Período Temporal

```
python
# Agrupar por mês
agrupado_mes = df.groupby(df['Data'].dt.to_period('M')).agg({'Valor': 'sum'})

# Agrupar por trimestre
agrupado_trimestre = df.groupby(df['Data'].dt.to_period('Q')).agg({'Valor': 'mean'})

# Agrupar por semana
agrupado_semana = df.groupby(df['Data'].dt.to_period('W')).agg({'Valor': 'count'})
```

## 2. 🧹 Limpeza e Formatação de Dados

### 2.1 Tratamento de Strings

```
python
# Remover espaços em branco
df['Coluna'] = df['Coluna'].str.strip()           # Ambos os lados
df['Coluna'] = df['Coluna'].str.lstrip()         # Lado esquerdo
df['Coluna'] = df['Coluna'].str.rstrip()         # Lado direito

# Padronizar capitalização
df['Nome'] = df['Nome'].str.title()              # Primeira Letra Maiúscula
df['Nome'] = df['Nome'].str.upper()              # Tudo Maiúsculo
df['Nome'] = df['Nome'].str.lower()              # Tudo Minúsculo

# Substituir caracteres
df['Coluna'] = df['Coluna'].str.replace('old', 'new')

# Dividir strings
df[['Nome', 'Sobrenome']] = df['Nome_Completo'].str.split(' ', 1, expand=True)
```

### 2.2 Tratamento de Valores Nulos

```
python
# Identificar valores nulos
nulos_por_coluna = df.isnull().sum()
percentual_nulos = df.isnull().mean() * 100

# Remover valores nulos
```

```

df_sem_nulos = df.dropna() # Remove todas as linhas com nulos
df_sem_nulos_coluna = df.dropna(subset=['Coluna']) # Remove nulos de coluna
específica

# Preencher valores nulos
df['Coluna'] = df['Coluna'].fillna(0) # Preenche com 0
df['Coluna'] = df['Coluna'].fillna('Desconhecido') # Preenche com texto
df['Coluna'] = df['Coluna'].fillna(method='ffill') # Preenche com valor anterior
df['Coluna'] = df['Coluna'].fillna(method='bfill') # Preenche com próximo valor
df['Coluna'] = df['Coluna'].fillna(df['Coluna'].mean()) # Preenche com média

```

## 2.3 Tratamento de Duplicatas

```

python
# Identificar duplicatas
duplicatas = df.duplicated() # Linhas completamente duplicadas
duplicadas_coluna = df.duplicated(subset=['Coluna']) # Duplicatas em coluna
específica

# Remover duplicatas
df_sem_duplicatas = df.drop_duplicates() # Remove todas as duplicatas
df_sem_duplicatas_coluna = df.drop_duplicates(subset=['Coluna'])

# Manter primeira/última ocorrência
df_primeira = df.drop_duplicates(keep='first')
df_ultima = df.drop_duplicates(keep='last')

```

## 2.4 Conversão de Tipos de Dados

```

python
# Converter para numérico
df['Coluna'] = pd.to_numeric(df['Coluna'], errors='coerce')

# Converter para categoria (eficiente para strings repetidas)
df['Categoria'] = df['Coluna'].astype('category')

# Converter datas (como já mostrado)
df['Data'] = pd.to_datetime(df['Data_Texto'])

# Converter booleanos
df['Booleano'] = df['Coluna'].astype(bool)

```

## 2.5 Criação e Transformação de Colunas

```
python
# Criar colunas baseadas em condições
df['Classificacao'] = np.where(df['Valor'] > 100, 'Alto', 'Baixo')

# Usar apply para transformações complexas
def categorizar(valor):
    if valor < 30:
        return 'Jovem'
    elif valor < 60:
        return 'Adulto'
    else:
        return 'Senior'

df['Faixa'] = df['Idade'].apply(categorizar)
```

```
python
# Formatação monetária
df['Valor_Formatado'] = df['Valor'].apply(lambda x: f'R$ {x:,.2f}')

# Formatação percentual
df['Percentual_Formatado'] = df['Percentual'].apply(lambda x: f'{x:.1%}')

# Formatação de datas
df['Data_Formatada'] = df['Data'].dt.strftime('%d/%m/%Y')
df['Data_Hora_Formatada'] = df['Data_Hora'].dt.strftime('%d/%m/%Y %H:%M')
```

## 2.6 Formatação de Valores

```
python

# Formatação monetária
df['Valor_Formatado'] = df['Valor'].apply(lambda x: f'R$ {x:,.2f}')

# Formatação percentual
df['Percentual_Formatado'] = df['Percentual'].apply(lambda x: f'{x:.1%}')

# Formatação de datas
df['Data_Formatada'] = df['Data'].dt.strftime('%d/%m/%Y')

df['Data_Hora_Formatada'] = df['Data_Hora'].dt.strftime('%d/%m/%Y %H:%M')
```

### 3. Exportação para Excel

#### 3.1 Exportação Básica

```
python
# Exportar DataFrame para Excel
df.to_excel('arquivo.xlsx', index=False, engine='openpyxl')

# Com parâmetros adicionais
df.to_excel(
    'arquivo.xlsx',
    index=False,                # Não incluir índice
    sheet_name='Dados',        # Nome da planilha
    na_rep='N/A',              # Representação para valores nulos
    float_format='%.2f',       # Formato para números decimais
    engine='openpyxl'          # Motor para .xlsx
)
```

#### 3.2 Múltiplas Planilhas

```
python
# Criar arquivo com múltiplas planilhas
with pd.ExcelWriter('relatorio.xlsx', engine='openpyxl') as writer:
    # Planilha 1: Dados completos
    df_completo.to_excel(writer, sheet_name='Dados_Completos', index=False)

    # Planilha 2: Resumo
    df_resumo.to_excel(writer, sheet_name='Resumo', index=False)

    # Planilha 3: Análises
    df_analise.to_excel(writer, sheet_name='Analises')

# Ajustar largura das colunas (opcional, requer openpyxl)
worksheet = writer.sheets['Dados_Completos']
for column in worksheet.columns:
    max_length = 0
    column_letter = column[0].column_letter
    for cell in column:
        try:
            if len(str(cell.value)) > max_length:
                max_length = len(str(cell.value))
        except:
            pass
```

```
        pass
    adjusted_width = (max_length + 2)
    worksheet.column_dimensions[column_letter].width = adjusted_width
```

### 3.3 Exportação com Formatação Avançada

```
python
# Preparar dados formatados
df_exportacao = df.copy()

# Aplicar formatações
df_exportacao['Valor_Formatado'] = df_exportacao['Valor'].apply(
    lambda x: f'R$ {x:,.2f}' if pd.notnull(x) else ''
)

df_exportacao['Data_Formatada'] = df_exportacao['Data'].apply(
    lambda x: x.strftime('%d/%m/%Y') if pd.notnull(x) else ''
)

# Remover colunas originais se necessário
df_exportacao = df_exportacao.drop(['Valor', 'Data'], axis=1)

# Exportar
df_exportacao.to_excel('dados_formatados.xlsx', index=False, engine='openpyxl')
```

### 3.4 Exportação de Dados Agrupados

```
python
# Criar diferentes análises
analise_1 = df.groupby('Categoria').agg({
    'Valor': ['count', 'mean', 'sum', 'min', 'max']
}).round(2)

analise_2 = df.groupby(['Categoria', 'Subcategoria']).agg({
    'Valor': 'sum'
}).reset_index()

analise_3 = df.pivot_table(
    index='Categoria',
    columns='Mês',
    values='Valor',
    aggfunc='sum'
```

```

).fillna(0)

# Exportar todas as análises
with pd.ExcelWriter('analises.xlsx', engine='openpyxl') as writer:
    analise_1.to_excel(writer, sheet_name='Resumo_Categoria')
    analise_2.to_excel(writer, sheet_name='Detalhado', index=False)
    analise_3.to_excel(writer, sheet_name='Pivot')

```

## 3.5 Boas Práticas na Exportação

```

python
# 1. Sempre especificar o engine
df.to_excel('arquivo.xlsx', engine='openpyxl')

# 2. Evitar exportar o índice (a menos que seja necessário)
df.to_excel('arquivo.xlsx', index=False, engine='openpyxl')

# 3. Usar nomes significativos para as planilhas
df.to_excel('arquivo.xlsx', sheet_name='Clientes_2024', index=False,
engine='openpyxl')

# 4. Incluir timestamp no nome do arquivo para versionamento
from datetime import datetime
timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
nome_arquivo = f'relatorio_{timestamp}.xlsx'
df.to_excel(nome_arquivo, index=False, engine='openpyxl')

# 5. Documentar o conteúdo das planilhas
with pd.ExcelWriter('relatorio_completo.xlsx', engine='openpyxl') as writer:
    df.to_excel(writer, sheet_name='Dados_Brutos', index=False)

    # Adicionar uma planilha de documentação
    doc_data = {
        'Informação': ['Data de geração', 'Fonte dos dados', 'Última atualização'],
        'Valor': [datetime.now().strftime('%d/%m/%Y %H:%M'), 'Sistema Clínica',
'Hoje']
    }
    pd.DataFrame(doc_data).to_excel(writer, sheet_name='Documentação', index=False)

```

## 3.6 Tratamento de Erros na Exportação

```
python
```



```
try:
    df.to_excel('relatorio.xlsx', index=False, engine='openpyxl')
    print("✅ Arquivo exportado com sucesso!")

except PermissionError:
    print("❌ Erro: O arquivo está aberto em outro programa.")
    print("    Feche o arquivo e tente novamente.")

except Exception as e:
    print(f"❌ Erro inesperado: {e}")
    print("    Verifique se o pandas e openpyxl estão instalados:")
    print("    pip install pandas openpyxl")
```

---

## Dicas Práticas

### Para Datas:

1. Sempre converta strings para datetime o quanto antes
2. Use `errors='coerce'` para identificar problemas nos dados
3. O método `dt` só funciona com colunas do tipo datetime

### Para Limpeza:

1. Sempre faça uma cópia antes de modificar dados originais
2. Use `inplace=True` com cuidado (é melhor reatribuir)
3. Documente todas as transformações aplicadas

### Para Excel:

1. Use `engine='openpyxl'` para arquivos .xlsx
2. Para .xls, use `engine='xlrd'` (mas xlrd não suporta .xlsx)
3. Arquivos grandes podem ser exportados em partes

### Performance:

```
python
# Para grandes volumes de dados
import pandas as pd

# Leitura eficiente
df = pd.read_excel('arquivo.xlsx',
                   engine='openpyxl',
                   usecols=['col1', 'col2'], # Ler apenas colunas necessárias
                   nrows=1000)              # Ler apenas N linhas para teste

# Exportação eficiente
df.to_excel('saida.xlsx',
            index=False,
            engine='openpyxl',
            chunksize=10000)                # Exportar em blocos
```

---

## Checklist de Validação

### Antes da Exportação:

- Todas as datas estão no tipo datetime
- Valores nulos foram tratados apropriadamente
- Tipos de dados estão corretos (números como numéricos, etc.)
- Nomes de colunas estão claros e sem caracteres especiais
- Dados duplicados foram removidos quando necessário
- Formatações foram aplicadas consistentemente

### Após a Exportação:

- O arquivo abre sem erros no Excel
- Todas as planilhas estão presentes
- Formatações foram mantidas
- Valores nulos estão representados corretamente
- O tamanho do arquivo é razoável

Este documento serve como referência rápida para as operações mais comuns com pandas. Guarde-o e consulte sempre que necessário!

